United International University (UIU)
Dept. of Computer Science and Engineering (CSE)
CSE 1116, Object Oriented Programming Lab
Section I, Summer 2024, MoI
Total Marks: 15

Mid Replacement Assignment

Question 1: (Array and String with Input, Class) ***[5]***

Console Input (2)	Array (1)	Logic (2)

Write a program which will take employee information of an office. Where the number of employees will be provided by console as **n**. After this number, you will get names and ids and salaries of employees. Create a class **Employee** and store the information received from the console in an array of employees. Now, traverse and find the total number of employees who have a salary of at least 12000 or more, also print the maximum employee salary. Please check the following sample input/output along with the explanation.

Input	Output	Explanation
4 Forhad Hossain FH-101 10000 Emran Ahmed EA-102 20000 Riad Hasan RdH-103 15500 Sadia Tabassum SaTb-104 21000	Count: 3 Max = 21000	Emran, Riad and Sadia Have salaries >= 12000 and maximum salary is 21000

Question 2: (Method Overloading, Encapsulation, Polymorphism, Abstraction) ***[10]***

Sendable interface(1)	Message(1)	TextMessage(2)	AudioMessage(2)	ImageMessage(2)	Main(2)

You are building a messaging application that supports various types of messages. At first it includes text messages and audio messages. The system has an abstract class called "**Message"** and an interface called "**Sendable**."

The "**Message" abstract class** has the following attributes and methods:

<u>*Attributes*</u> - **sender** (string): The sender of the message. *[The subclasses need access to the* ***sender*** *attribute to set it during their own object construction.]* **timestamp** (string): The timestamp when the message was sent. **receiver** (string): The receiver of the message. *[The subclasses need access to the* ***receiver*** *attribute to set it during their own object construction.]*

<u>*Methods:*</u>

get_details(): string: an abstract method; Returns a string containing the sender, receiver and timestamp of the message.

send(): void: a concrete method; Sends the message.

The "**Sendable" interface** defines the following methods:

<u>*Methods:*</u>

prepare(): void: Prepare different type of messages for sending.

deliver(): void: Deliver different type of messages to the recipient.

You are tasked with creating two concrete classes: "***TextMessage," and "AudioMessage***." All classes should inherit from the "**Message**" class and have the feature of "**Sendable**" interface.

The "***TextMessage"*** class has the following additional attributes:

<u>*Attributes*</u> **- content** (string): The content of the text message. *[can only be accessed and modified within the* ***TextMessage*** *class itself.]*

<u>*Methods:*</u> **- get_details(): string**: show the sender, timestamp, receiver and the content of the text message.

prepare(): void: prepare the text message for sending.

deliver(): void: to deliver the text message to the recipient.

The "***AudioMessage***" class has the following additional attributes:

<u>*Attributes*</u> **- audio_url** (string): The URL of the audio file. *[it's not directly accessible to subclasses.]*

<u>*Methods*</u> **- get_details(): string**: show the sender, timestamp, receiver and URL of the audio file.

prepare(): void: prepare the audio message for sending.

deliver(): void: deliver the audio message to the recipient.

Imagine that you need to extend the messaging application to support a new type of message called "***ImageMessage"*** The "***ImageMessage"*** class should have the feature of "Sendable" interface to enable sending and delivery of image messages.

The "***ImageMessage"*** class has the following additional attributes:

Attributes - **image_url** (string): The URL of the image which can only be accessed and modified within the
ImageMessage class itself.

Methods - **get_details(): string**: return the sender, timestamp, receiver and image URL of the image.
prepare(): void: prepare the image message for sending.
deliver(): void: deliver the image message to the recipient.

Your task is to implement* all the parent class, interface and subclasses *classes according to the given specifications. Additionally, write a program that demonstrates the usage of the classes by creating instances of messages, invoking the appropriate methods, and simulating the sending and delivery of messages.

Feel free to add additional methods or attributes to the classes to further enhance the functionality of the messaging application.

Sample Output:

For the text message :
Sender: Borno, Timestamp: 2023-08-13T06:31:06.592955600, Receiver: Rahi, Content: Hello, how are you?
Preparing text message for sending...
Delivering text message to the recipient Rahi...

For the Image message :
Sender: Pronoy, Timestamp: 2023-08-13T06:31:06.608578100, Receiver: Borno, Image URL: https://example.com/image.jpg
Preparing image message for sending...
Delivering image message to the recipient Borno...

For the Audio Message
Sender: Rahi, Timestamp: 2023-08-13T06:31:06.608578100, Receiver: Pronoy, Audio URL: https://example.com/audio.mp3
Preparing audio message for sending...
Delivering audio message to the recipient Pronoy…